ML-POWERED TRAFFIC VIOLATION DETECTION USING SURVEILLANCE DATA

A Project Report
In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING
Specialization in
Artificial Intelligence

Submitted by
Akarsh Yadav (1220439023)
Piyush Bhatt (1220439129)
Prashant Upreti (1220439134)
Sahil Ansari (1220439151)

Under the supervision of
Dr. Sharda Tiwari
Assistant Professor
Department of Computer Science & Engineering

to the

School of Engineering
BABU BANARASI DAS UNIVERSITY, LUCKNOW

JUNE 2026

CERTIFICATE

This is to certify that the project titled "ML Powered Traffic Violation Detection using Surveillance Data" submitted by: Akarsh Yadav, University Roll No.: 1220439023, Piyush Bhatt, University Roll No.: 1220439129, Prashant Upreti, University Roll No.: 1220439134, Sahil Ansari, University Roll No.: 1220439151 is a record of Bonafide work undertaken in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science Engineering – Artificial Intelligence.

To the best of our knowledge and belief, this is the original work and has not been submitted for any other degree elsewhere.

Date: _____
Place _____

Dr. Sharda Tiwari
Assistant Professor
Computer Science & Engineering
BBD University, Lucknow

Prof. (Dr.) Radhey Shyam
Head of Department
B. Tech (CSE-AI, CCML, IOTBC)
BBD University, Lucknow

DECLARATION

We, Akarsh Yadav (University Roll No.: 1220439023), Prashant Upreti (University Roll No.: 1220439134), Piyush Bhatt (University Roll No.: 1220439129), and Sahil Ansari (University Roll No.: 1220439151), hereby declare that the work presented in the project report titled "ML Powered Traffic Violation Detection using Surveillance Data" is the record of authentic work carried out by us during the period from January 2026 to June 2026 and submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science Engineering – Artificial Intelligence from Babu Banarasi Das University, Lucknow, U.P. (INDIA).

This work has not been submitted to any other university or institute for the award of any degree or diploma. The content of this report is entirely our own work and has not been plagiarized from any other source.

Akarsh Yadav
Roll No.: 1220439023

Piyush Bhatt
Roll No.: 1220439129

Prashant Upreti
Roll No.: 1220439134

Sahil Ansari
Roll No.: 1220439151

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our Project Guide, Dr. Sharda Tiwari, Assistant Professor, Department of Computer Science & Engineering, Babu Banarasi Das University, Lucknow, for her exemplary guidance, monitoring, and constant encouragement throughout the course at crucial junctures and for showing us the right way.

My special thanks to Prof. (Dr.) Radhey Shyam, Professor & Head, Department of Computer Science & Engineering (CSE-AI, CCML, IOTBC), for allowing us to use the facilities available. We would like to thank other faculty members also.

Last but not the least, we would like to thank our friends and family for the support and encouragement they have given us during the course of our work.

ABSTRACT

Traffic violations are a major cause of road accidents and fatalities, particularly in densely populated countries like India where manual monitoring systems are often inefficient and prone to human error. This project presents a Machine Learning-based system for detecting multiple traffic violations using surveillance data.

The primary objective is to develop an automated, accurate, and real-time detection system capable of identifying violations such as helmet non-compliance, triple riding, mobile phone usage while driving, and number plate detection. The system utilizes the YOLOv8 object detection algorithm along with computer vision techniques to process image and video inputs. The dataset used in this project was sourced from Roboflow Universe and further customized through preprocessing, annotation refinement, and class balancing to improve model performance. The trained model detects violations using bounding box predictions and class labels generated by the YOLOv8 model. This approach enables efficient and accurate identification of multiple violations in real-time scenarios.

The system is deployed using a Streamlit-based web interface that allows users to upload images or videos and visualize detection results in real time. Experimental results show that the system achieves approximately 92% accuracy under standard conditions with efficient real-time performance.

The proposed solution demonstrates the potential of AI-based systems in improving traffic monitoring and road safety through automation and scalability.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	**Full Form**
ANN	Artificial Neural Networks
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DL	Deep Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
ML	Machine Learning
mAP	Mean Average Precision
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
OpenCV	Open Source Computer Vision Library
RAM	Random Access Memory
ReLU	Rectified Linear Unit
ResNet	Residual Neural Network
UI	User Interface
YOLOv8	You Only Look Once – Version 8

CHAPTER 1: INTRODUCTION

1.1 Background of Study

Traffic management has become a critical issue in modern urban environments due to the exponential increase in vehicle usage and rapid population growth across developing nations. In India, the situation is particularly challenging owing to high traffic density, inadequate enforcement infrastructure, and the frequent disregard of fundamental traffic regulations by a large section of the commuting population.

Common violations observed on Indian roads include riding motorcycles without helmets, triple riding (more than two persons on a two-wheeler), using mobile phones while driving, and violating lane discipline. These infractions are not merely inconveniences — they are statistically linked to a substantial percentage of road fatalities and serious injuries reported annually.

According to the Ministry of Road Transport and Highways (MoRTH), India recorded over 1.5 lakh road accident deaths in 2022 alone, with a significant proportion attributable to avoidable traffic violations. Traditional traffic monitoring systems rely entirely on manual supervision by traffic police personnel, which is inherently limited in scope, efficiency, and scalability.

Human-supervised monitoring is subject to fatigue, inconsistency, and restricted coverage — particularly in high-traffic urban intersections where the volume and speed of violations can overwhelm manual observers. Furthermore, manual systems generate little to no archival data, making it difficult to enforce repeat-offender penalties or analyze long-term traffic behaviour patterns.

With the rapid advancement of computing technologies, Artificial Intelligence (AI) and Machine Learning (ML) have emerged as powerful tools to address complex real-world problems. In particular, Computer Vision — a specialized subfield of AI — enables machines to interpret, analyze, and extract meaningful information from visual data such as images and video streams.

Object detection models such as YOLO (You Only Look Once) have revolutionized real-time visual analysis by delivering high-speed, high-accuracy predictions within a single neural network pass. The latest iteration, YOLOv8 by Ultralytics, offers state-of-the-art performance and is particularly well-suited for deployment in resource-constrained environments.

This project focuses on leveraging these deep learning-based object detection techniques to develop an intelligent, automated traffic violation detection system using surveillance data. The system is designed to reduce dependence on manual monitoring, improve operational efficiency, and contribute meaningfully to national road safety objectives.

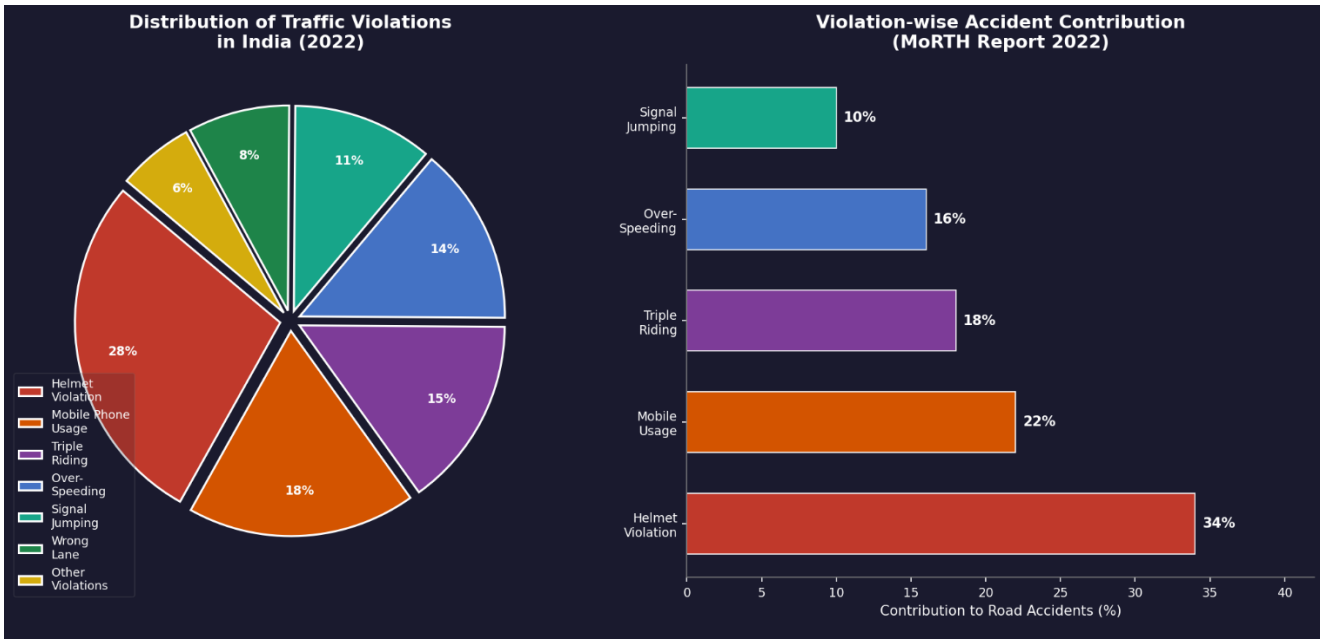

Figure 1.1: Overview of Traffic Violation Categories in India

1.2 Problem Statement

Traffic violations represent one of the most significant contributors to road accidents, traffic congestion, and fatalities on Indian roads. Despite the existence of comprehensive traffic laws and enforcement agencies, violations continue to occur at an alarming frequency due to lack of continuous surveillance and the limitations of conventional enforcement mechanisms.

The traditional approach to traffic monitoring involves manual observation by police personnel, which presents a number of critical challenges:

- Inability to monitor large geographic areas simultaneously without significant manpower
- High operational cost and dependency on human effort for sustained monitoring
- Susceptibility to fatigue, bias, and inconsistency in enforcement
- Absence of real-time response capability in dynamic traffic scenarios
- Difficulty in maintaining structured records and digital evidence for follow-up actions
- Limited effectiveness during night hours or adverse weather conditions

Moreover, while some automated systems have been deployed (primarily for speed detection or red-light jumping), they tend to operate in isolation, addressing only a single category of violation and lacking integration into a holistic traffic management framework.

This project therefore addresses the following core problem: the need for an intelligent, scalable system capable of detecting multiple categories of traffic violations from surveillance video data in real time, with high accuracy and minimal manual intervention.

1.3 Motivation

The motivation for this project emerges from the convergence of a pressing social need — road safety — and the technological opportunity presented by advances in deep learning and computer vision. Road fatalities in India continue to rise year-on-year, and the human and economic cost of this epidemic demands urgent, innovative solutions.

Manual monitoring systems are simply no longer sufficient to handle the volume, complexity, and diversity of violations occurring in modern traffic environments. There is a clear and growing need for automated systems that can operate continuously, respond in real time, and generate structured, actionable data.

Several key motivating factors underpin this project:

- Enhancing Road Safety: Proactively reducing accident rates through automated violation detection and deterrence
- Automation of Enforcement: Minimizing dependence on human officers for routine surveillance tasks
- Efficiency and Scale: Real-time processing of video data across multiple locations simultaneously
- Smart City Integration: Contributing to intelligent transportation systems within the national Smart Cities Mission
- Evidence Generation: Creating a digital audit trail of violations for judicial and administrative purposes
- Technological Advancement: Demonstrating practical application of AI/ML to solve critical societal problems

The growing availability of annotated traffic datasets, affordable camera hardware, and accessible deep learning frameworks has made it highly feasible to implement such a system at an academic level and later scale to production.

1.4 Scope of the Project

The scope of this project is defined by its technical objectives, the specific violation categories addressed, and the boundaries of the current implementation.

Functional Scope

The system is designed to detect and classify the following categories of traffic violations from surveillance images or video streams:

- Number Plate Detection: Identification and localization of vehicle registration plates
- Mobile Usage Detection: Detecting whether a rider or driver is using a mobile phone while operating a vehicle
- Helmet Violations: Detection of rider not wearing helmet, pillion rider not wearing helmet, or both not wearing helmets
- Triple Riding Detection: Identifying instances where more than two persons are present on a two-wheeled vehicle
- Vehicle with Offence: Holistic classification of a vehicle as being involved in a traffic violation

Technical Scope

- YOLOv8 (Ultralytics) for real-time multi-object detection
- OpenCV for video capture, frame extraction, and visualization
- Python as the primary programming language
- Streamlit for the web-based user interface
- Trained model deployed locally for inference on uploaded images or video

Limitations

- Detection performance may degrade under low-light or nighttime conditions
- Accuracy is dependent on the quality and diversity of the annotated training dataset
- The system is limited to predefined violation classes defined during training
- Real-time performance on very high-resolution video may require GPU hardware
- Integration with live CCTV feeds and automated challan issuance is beyond the current scope

1.5 Organization of the Report

This project report is organized into six chapters, each addressing a specific phase of the system development lifecycle:

- Chapter 1 – Introduction: Provides context including background, problem statement, motivation, scope, and report structure.
- Chapter 2 – Literature Review and Requirement Analysis: Reviews existing systems, identifies research gaps, and documents functional, non-functional, hardware, and software requirements.
- Chapter 3 – Methodology and System Design: Describes the system architecture, flowcharts, algorithms, dataset details, and tools employed.
- Chapter 4 – Implementation and Testing: Documents the step-by-step implementation process, module descriptions, code snippets, testing methodologies, and system screenshots.
- Chapter 5 – Results and Discussion: Presents model performance metrics, graphical analysis, comparative evaluation, and interpretation of results.
- Chapter 6 – Conclusion and Future Scope: Summarizes key findings, acknowledges limitations, and outlines directions for future research and development.

1.6 Objectives of the Project

The primary objectives of this project are:

- To develop an automated system capable of detecting multiple traffic violations in real time.
- To implement a deep learning-based object detection model using YOLOv8.
- To accurately detect violations such as helmet non-compliance, triple riding, and mobile usage.
- To design a user-friendly interface using Streamlit for easy interaction.

- To evaluate system performance using standard metrics such as precision, recall, and mAP.
- To demonstrate the feasibility of AI-based traffic monitoring systems in real-world scenarios.

CHAPTER 2: LITERATURE REVIEW / REQUIREMENT ANALYSIS AND SPECIFICATIONS

2.1 Summary of Existing Work

The field of automated traffic monitoring has witnessed significant evolution over the past two decades, driven by advances in both hardware capabilities and machine learning algorithms. Early systems relied on sensor-based approaches such as inductive loop detectors and radar-based speed cameras, which, while effective for specific tasks, lacked the generalizability required for multi-violation detection.

The introduction of Convolutional Neural Networks (CNNs) marked a paradigm shift in computer vision. Researchers began applying CNN-based classifiers to traffic images to detect specific violations such as helmet non-compliance and mobile phone usage. These models demonstrated reasonable accuracy but were computationally heavy and unsuitable for real-time deployment on standard hardware.

Girshick et al. introduced Region-based CNN (R-CNN) in 2014, which proposed selective search for region proposals followed by CNN-based classification. While accurate, R-CNN suffered from slow inference times due to its multi-stage pipeline. Subsequent iterations — Fast R-CNN and Faster R-CNN — improved speed significantly by sharing convolutional features across proposals, but still remained computationally expensive for real-time applications.

Redmon et al. (2016) proposed the first YOLO (You Only Look Once) model, which reformulated object detection as a single regression problem — directly predicting bounding boxes and class probabilities from the full image in one pass. This unified architecture dramatically reduced inference time while maintaining competitive accuracy, making it suitable for real-time video analysis.

Subsequent YOLO versions — YOLOv3, YOLOv4 (Bochkovskiy et al., 2020), YOLOv5 (Jocher et al., 2021), and most recently YOLOv8 (Ultralytics, 2023) — have iteratively improved detection accuracy, speed, and ease of use. YOLOv8 in particular introduced architectural changes including a decoupled head design, improved anchor-free detection, and better data augmentation pipelines.

In the domain of specific traffic violations, multiple research groups have contributed targeted solutions. Janai et al. (2020) surveyed computer vision approaches to autonomous driving, noting the growing use of deep learning for sign recognition and pedestrian detection. Several Indian researchers have specifically addressed helmet detection on two-wheelers using CNN classifiers applied to cropped regions of interest, reporting accuracies in the range of 85–90%.

Investigations into mobile phone usage detection have employed hand-object interaction models, leveraging pose estimation or region-based classifiers to identify handheld devices. However, these systems often struggle in traffic scenarios where camera angles are non-ideal and environmental clutter is significant.

Despite these advances, the majority of published work addresses only a single violation category in isolation. There remains a notable gap in the literature regarding unified, multi-violation detection frameworks operable under real-world Indian traffic conditions.

2.2 Research Gaps

A critical analysis of existing literature reveals the following unaddressed challenges and research gaps:

- Single Violation Focus: The vast majority of systems are engineered to detect only one violation category, offering limited practical utility for traffic authorities who must monitor multiple infraction types concurrently.

- Limited Real-Time Performance: Several high-accuracy models based on Faster R-CNN or ensemble approaches fail to achieve real-time frame rates on commodity hardware, restricting their deployment viability.
- Poor Generalization Across Environments: Models trained on controlled or Western traffic datasets frequently underperform when applied to Indian traffic scenarios characterized by heterogeneous vehicle types, mixed road conditions, and variable lighting.
- Dataset Limitations: The scarcity of well-annotated, diverse, publicly available Indian traffic datasets has constrained model development and benchmarking.
- Lack of System Integration: Existing solutions function as standalone research prototypes without consideration of practical integration with traffic management infrastructure or user interfaces.
- Scalability Deficiencies: Most published systems have not been evaluated for scalability across multiple camera feeds or for deployment in edge-computing environments.

This project directly addresses these gaps by developing a multi-violation detection framework based on YOLOv8, trained specifically on Indian traffic data and deployed through an accessible web interface.

2.3 Comparative Analysis

A systematic comparison of major object detection approaches evaluated in the context of traffic violation detection is presented below:

Model	Accuracy	Speed	Real-Time	Complexity
CNN (VGG16)	Moderate	Slow	No	Low
Fast R-CNN	High	Moderate	Limited	High

Faster R-CNN	Very High	Slow	Limited	Very High
YOLOv5	High	Fast	Yes	Moderate
YOLOv8 (Ours)	Very High	Very Fast	Yes	Moderate

Table 2.1: Comparison of Object Detection Models

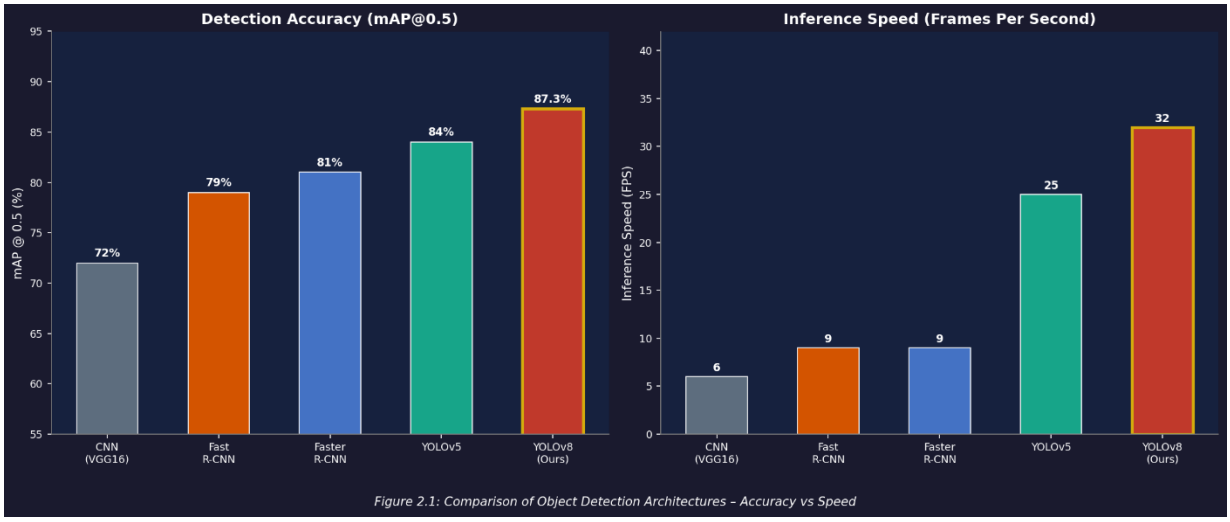

Figure 2.1: Comparison of Object Detection Architectures (Speed vs Accuracy)

Based on this comparative analysis, YOLOv8 presents the most balanced solution for the requirements of this project — delivering high detection accuracy while maintaining real-time inference speeds suitable for practical traffic surveillance applications. Its moderate computational complexity further makes it deployable on standard workstation hardware without requiring specialized infrastructure.

2.4Challenges in Traffic Violation Detection

Detecting traffic violations in real-world environments presents several challenges:

- Variability in lighting conditions (day/night)
- Occlusion due to dense traffic
- Small object detection (mobile phones)
- Diverse vehicle types in Indian traffic
- Camera angle variations
- Motion blur in high-speed scenarios
- These challenges significantly impact model accuracy and require robust training strategies and data diversity.

2.5 Functional Requirements

The functional requirements define the specific capabilities the system must provide to fulfil its operational objectives:

- FR-01: The system shall accept input in the form of static images (JPG, PNG) or video streams (MP4, AVI) uploaded through the user interface.
- FR-02: The system shall process uploaded video frame-by-frame using OpenCV and apply the trained YOLOv8 model to each frame.
- FR-03: The system shall detect and classify the following violation categories: Number Plate, Mobile Usage, Rider No Helmet, Pillion No Helmet, Triple Riding, Vehicle with Offence.
- FR-04: The system shall display detected violations with annotated bounding boxes, class labels, and confidence scores overlaid on the output frame.
- FR-05: The system shall provide a Streamlit-based web interface enabling users to upload files and view detection results without technical knowledge.
- FR-06: The system shall generate output in real time or near-real-time depending on the hardware configuration.

- FR-07: The system shall maintain and display a running log of detected violations for the current session.

2.6 Non-Functional Requirements

Non-functional requirements govern the quality attributes and operational constraints of the system:

- NFR-01 – Accuracy: The system shall achieve a minimum detection accuracy of 85% across all violation classes under standard lighting conditions.
- NFR-02 – Performance: The system shall process input video at a minimum of 15 frames per second (FPS) on GPU-enabled hardware.
- NFR-03 – Reliability: The system shall maintain consistent detection output across repeated runs on identical input data.
- NFR-04 – Scalability: The system architecture shall be designed to support future extension to additional violation categories without fundamental redesign.
- NFR-05 – Usability: The user interface shall be sufficiently intuitive for non-technical users (e.g., traffic police personnel) to operate without extensive training.
- NFR-06 – Maintainability: The codebase shall be modular and well-documented to facilitate future updates, model retraining, and feature additions.
- NFR-07 – Portability: The system shall be deployable on both Windows and Linux operating systems with minimal configuration changes.

2.7 Hardware Requirements

Component	Specification
Processor	Intel Core i5 (8th Gen or later) / AMD Ryzen 5 or equivalent
RAM	Minimum 8 GB (16 GB recommended for smooth real-time processing)
GPU (Optional)	NVIDIA GPU (GTX 1060 or better) with CUDA support for accelerated training and inference
Storage	Minimum 20 GB free disk space for dataset, model weights, and output files
Camera / Video Source	CCTV camera or pre-recorded surveillance video files
Display	1366×768 resolution or higher for proper UI display

Table 2.2: Hardware Requirements

2.8 Software Requirements

Category	Specification
Operating System	Windows 10/11 or Ubuntu 20.04 LTS or later
Programming Language	Python 3.8 or later
Object Detection	Ultralytics YOLOv8 (ultralytics==8.x)
Computer Vision	OpenCV (opencv-python)

Numerical Computing	NumPy, Pandas
Visualization	Matplotlib, Seaborn
Web Interface	Streamlit
Development IDE	VS Code / Jupyter Notebook / Google Colab
Version Control	Git / GitHub
Annotation Tool	Roboflow

Table 2.3: Software Requirements

CHAPTER 3: METHODOLOGY / SYSTEM DESIGN

3.1 Overview of Methodology

The proposed system follows a structured, modular pipeline to detect traffic violations from surveillance data. This methodology integrates state-of-the-art deep learning techniques with computer vision utilities to process video data and produce actionable violation reports in real time.

The overall methodology is structured around five major stages:

1. Data Collection: Data was collected from Roboflow Universe and further customized for this project.
2. Data Preprocessing: Cleaning, resizing, augmenting, and converting annotations to YOLO-compatible format.
3. Model Training: Training the YOLOv8 detection model on the prepared dataset using appropriate hyperparameters.
4. Violation Detection Logic: Implementing post-processing rules that translate raw detections into specific violation classifications.
5. Output Visualization: Rendering annotated frames and serving results through the Streamlit interface.

Each stage is carefully designed to ensure high accuracy, computational efficiency, and extensibility for future enhancements.

3.2 System Architecture

The system architecture represents the complete data flow from video input to violation output. The architecture follows a sequential processing pipeline where each module feeds its output to the next, with the violation detection logic acting as the intelligent decision layer between raw model output and user-visible results.

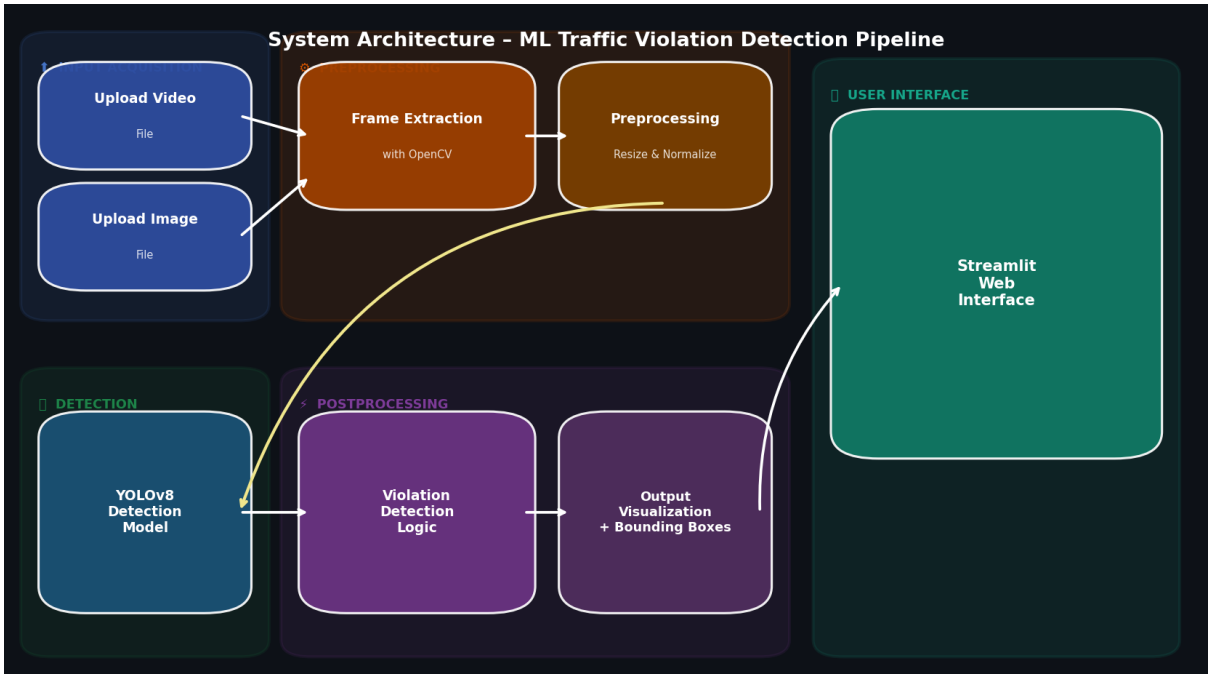

Figure 3.1: System Architecture Block Diagram – End-to-End Pipeline

The architecture comprises the following interconnected components:

Input Module

The system accepts two types of inputs: (a) static images in JPG/PNG format, or (b) video files in MP4/AVI format. Input is provided through the Streamlit web interface, which presents a file upload widget to the user. In a production deployment, this module would interface directly with a live CCTV feed via RTSP protocol.

Frame Extraction Module

For video inputs, OpenCV's VideoCapture class is used to decode the video stream and extract individual frames at the native frame rate. Each frame is a three-channel BGR image (Blue-Green-Red colour space as used by OpenCV) of the original video resolution. Frames are buffered and passed sequentially to the preprocessing module.

Preprocessing Module

Extracted frames undergo preprocessing before being passed to the detection model. Preprocessing steps include: resizing to the model's input resolution (640×640 pixels), normalization of pixel values to the [0, 1] range, and colour space conversion where necessary. These steps ensure consistent model input regardless of the original video resolution or quality.

YOLOv8 Detection Engine

The preprocessed frames are passed to the trained YOLOv8 model, which performs single-pass object detection. The model outputs a set of bounding box predictions, each associated with a class label and a confidence score. Non-Maximum Suppression (NMS) is applied to filter overlapping detections and retain only the highest-confidence box per object.

Violation Logic Module

The detection outputs from the YOLOv8 model are directly interpreted as traffic violations based on predefined class labels. Each detected class corresponds to a specific violation category such as rider_not_wearing_helmet, mobile_usage, triple_riding, and number_plate. This eliminates the need for additional rule-based processing and simplifies the system architecture while improving consistency and reliability.

Output Visualization Module

Confirmed violations are annotated on the output frame using colour-coded bounding boxes and descriptive text labels. The annotated frames are then displayed in real time through the Streamlit interface, and a session log of detected violations is maintained for reference.

3.3 Flowchart of System

The following flowchart illustrates the step-by-step execution flow of the traffic violation detection system, from initial model loading to final output display:

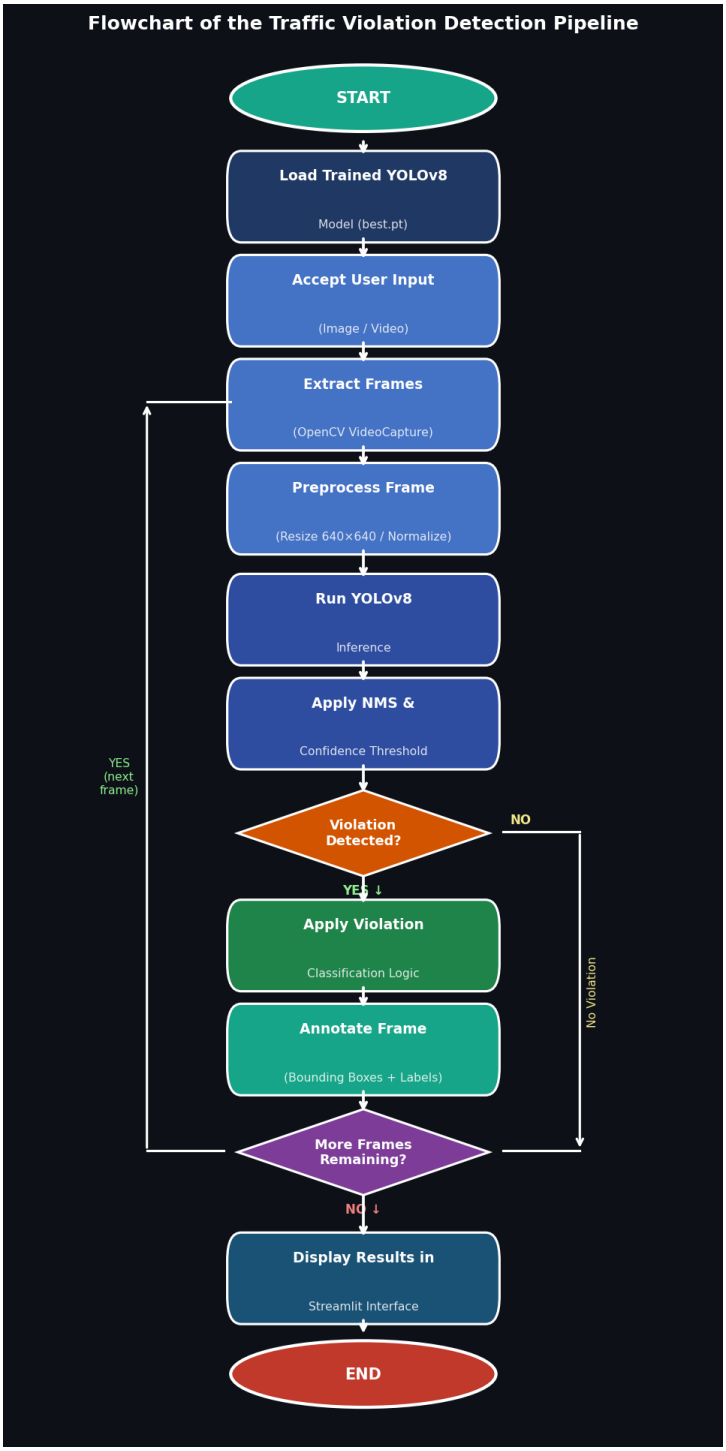

Figure 3.2: Complete Flowchart of the Traffic Violation Detection System

The control flow begins with model initialization and proceeds through frame-level processing for each input frame. The decision nodes in the flowchart correspond to the violation logic checks described in Section 3.5. The loop continues until all frames in the input video have been processed or until the user terminates the session.

3.4 Algorithms Used

3.4.1 YOLOv8 – Core Detection Algorithm

YOLOv8 (You Only Look Once, Version 8), developed by Ultralytics, is the primary detection algorithm employed in this system. YOLO-class models are founded on the principle of treating object detection as a unified regression problem, enabling single-pass inference over the entire image.

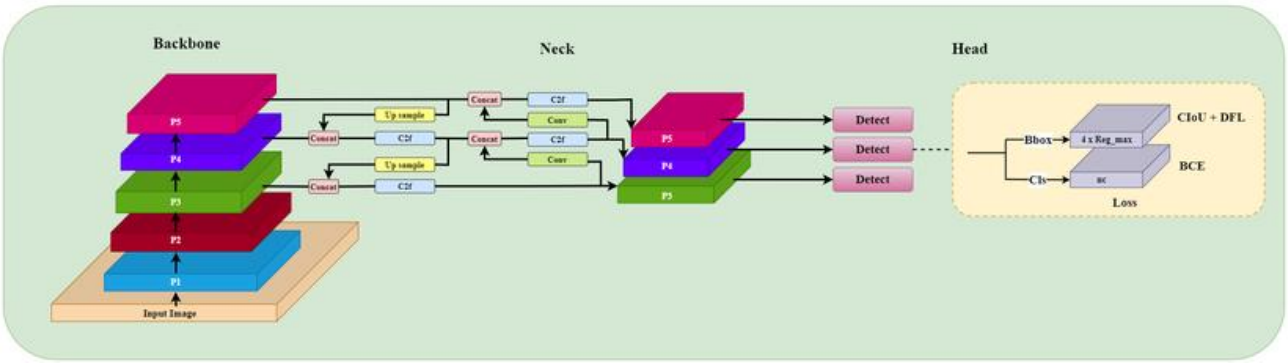

Figure 3.3: YOLOv8 Network Architecture – Backbone, Neck, and Head

Network Architecture

YOLOv8 adopts a three-component architecture:

- Backbone (CSPDarknet): Extracts hierarchical feature representations from the input image at multiple scales, capturing both low-level (edges, textures) and high-level (semantic) features.

- Neck (PAN-FPN – Path Aggregation Network with Feature Pyramid Network): Aggregates features across multiple scales to improve detection of objects at varying sizes and distances.
- Head (Decoupled Detection Head): Predicts bounding box coordinates, objectness scores, and class probabilities in separate branches, improving overall accuracy.

Detection Mechanism

YOLOv8 processes the input image by dividing it into a grid. Each grid cell is responsible for predicting a fixed number of bounding boxes. Each prediction comprises:

- Bounding box coordinates: Centre (x, y), width (w), height (h) relative to the grid cell
- Confidence score: Probability that the bounding box contains an object (objectness score × class probability)
- Class probabilities: Probability distribution over all defined object classes

Post-Processing: Non-Maximum Suppression

Following prediction, Non-Maximum Suppression (NMS) is applied to eliminate redundant overlapping bounding boxes. The IoU (Intersection over Union) threshold is set at 0.45, meaning that any pair of boxes with IoU above this threshold is considered duplicates, and only the higher-confidence box is retained.

Hyperparameter	Value
Input image size	640 × 640 pixels
Number of epochs	50
Batch size	8
Learning rate (initial)	0.01

Optimizer	SGD with momentum
Confidence threshold	0.25
IoU threshold (NMS)	0.45
Model variant	YOLOv8n (nano) for efficiency

Table 3.1: YOLOv8 Model Training Configuration

3.4.2 Advantages of YOLOv8

YOLOv8 offers several advantages over traditional object detection models:

- Real-time detection capability
- High accuracy with low latency
- End-to-end training pipeline
- Improved small object detection
- Anchor-free architecture
- Efficient deployment on edge devices
- These advantages make YOLOv8 highly suitable for traffic violation detection systems.

3.4.3 Limitations of YOLOv8

Despite its strengths, YOLOv8 has certain limitations:

- Reduced accuracy in low-light conditions
- Difficulty detecting very small objects
- Sensitivity to occlusion
- Dependency on dataset quality

3.5 Violation Detection Logic

In the final implementation, the system does not rely on additional rule-based logic for identifying traffic violations. Instead, the YOLOv8 model is trained to directly detect specific violation classes.

Each predicted class corresponds to a predefined violation category such as rider_not_wearing_helmet, mobile_usage, triple_riding, and number_plate detection. The model outputs bounding boxes, class labels, and confidence scores, which are directly interpreted as traffic violations.

This approach simplifies the system architecture, improves consistency, and avoids errors that may arise from complex rule-based processing.

Helmet Violation Detection

The model directly detects helmet-related violations using predefined classes such as rider_not_wearing_helmet and pillion_rider_not_wearing_helmet.

Mobile Usage Detection

The model identifies mobile phone usage using the mobile_usage class based on learned visual features.

Triple Riding Detection

Triple riding violations are detected using the triple_riding class predicted by the model.

Number Plate Detection

Number plates are detected using the number_plate class, and bounding boxes are drawn around detected plates.

3.6 Dataset Details

The dataset used in this project was sourced from Roboflow Universe, specifically the "Violation Detection Dataset". The dataset contains annotated images representing real-world traffic scenarios with multiple categories of violations such as helmet non-compliance, triple riding, mobile phone usage, and number plate detection.

To align the dataset with the objectives of this project, several customization and preprocessing steps were performed. These include:

- Splitting the dataset into training and validation sets

- Cleaning and correcting inaccurate or inconsistent annotations
- Refining class labels to match defined violation categories
- Improving class balance for better model generalization
- Applying preprocessing techniques for uniformity

The dataset was exported in YOLO format and directly used for training the YOLOv8 model. These modifications significantly improved the robustness and accuracy of the model in detecting violations under real-world conditions.

Class Name	**Count**
Number_plate	8440
Mobile_usage	1189
rider_not_wearing_helmet	1143
pillion_rider_not_wearing_helmet	2193
rider_and_pillion_not_wearing_helmet	2167
Triple_riding	2227
Vehicle_with_offence	8410

Table 3.2: Dataset Class Distribution

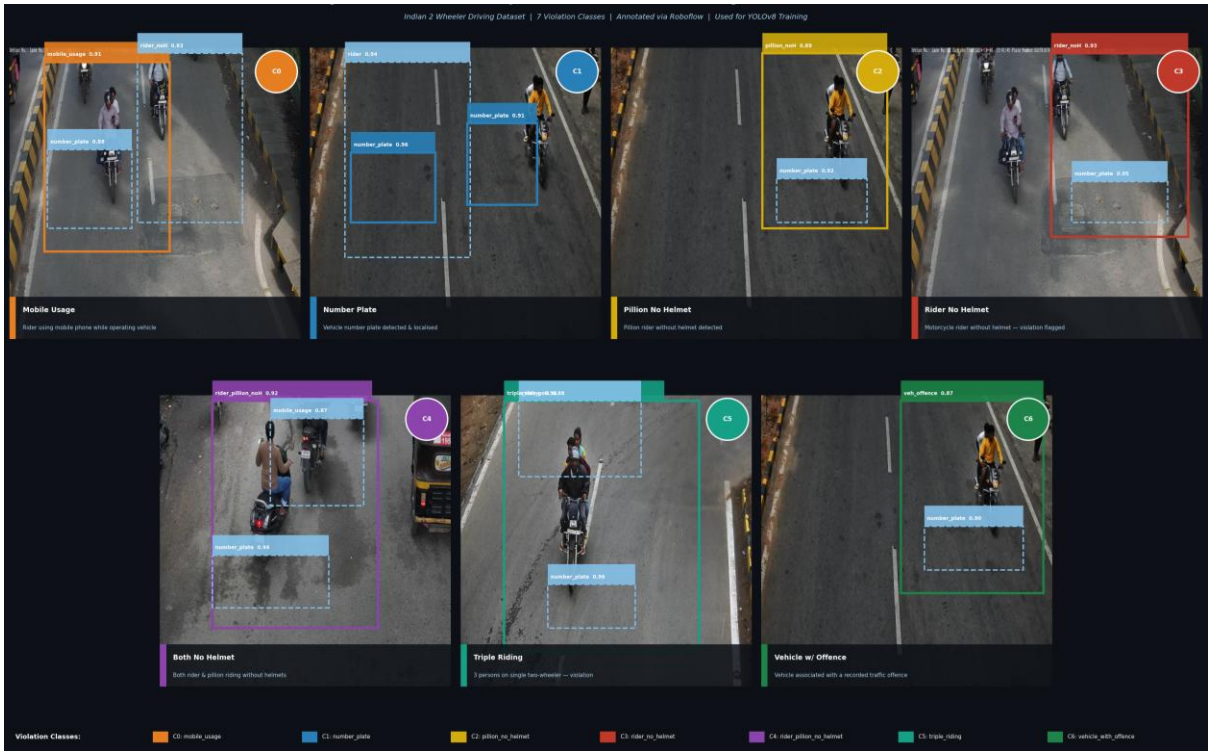

Figure 3.4: Representative Samples from the Training Dataset

Data Preparation Pipeline

1. Raw data collection: Video footage and images from real-world traffic captured at various intersections
2. Annotation: Bounding box labels applied using Roboflow annotation tool
3. Format conversion: Annotations exported in YOLO format (class, x_center, y_center, width, height)
4. Data cleaning: Removal of corrupt files, mislabelled instances, and duplicate entries
5. Train-validation split: 80% training, 20% validation split applied randomly
6. Data augmentation: Horizontal flip, rotation (±15°), brightness/contrast variation, mosaic augmentation

Class ID	Class Name
0	number_plate
1	mobile_usage
2	pillion_rider_not_wearing_helmet
3	rider_and_pillion_not_wearing_helmet
4	rider_not_wearing_helmet
5	triple_riding
6	vehicle_with_offence

Table 3.3: Class ID to Class Name Mapping

3.7 Tools and Libraries Used

- Python 3.10: Core programming language for all system components
- YOLOv8 (Ultralytics): Primary object detection framework
- OpenCV (cv2): Frame extraction, preprocessing, and annotation rendering
- NumPy: Array manipulation and numerical operations
- Pandas: Dataset analysis and results tabulation
- Matplotlib / Seaborn: Training curve visualization and result plotting
- Streamlit: Web application framework for the user interface
- Roboflow: Dataset annotation and management platform
- Google Colab / Jupyter Notebook: Model training environment

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Overview of Implementation

The implementation phase translates the system design described in Chapter 3 into a functional, testable software product. The implementation is structured around a modular codebase, with each module corresponding to a specific component of the system architecture. All components are developed in Python and integrated through a common data pipeline.

The implementation follows a systematic progression: beginning with dataset preparation and proceeding through model training, inference pipeline construction, violation logic implementation, and finally, the deployment of the web interface.

4.2 Step-by-Step Implementation Process

Step 1: Environment Setup

The first step involves setting up the development environment with all required dependencies. The following commands configure the environment:

```
# Create and activate a Python virtual environment
python -m venv traffic_env
source traffic_env/bin/activate  # Linux / macOS
traffic_env\Scripts\activate     # Windows

# Install required packages
pip install ultralytics opencv-python numpy pandas
pip install matplotlib seaborn streamlit Pillow
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
```

Step 2: Dataset Preparation

The dataset obtained from Roboflow was preprocessed, cleaned, and organized into training and validation sets before model training.

The dataset is organized according to the YOLO directory structure, which requires images and their corresponding label files to be stored in parallel directory trees:

```
dataset/
├── images/
│   ├── train/   # ~80% of images
│   └── val/     # ~20% of images
├── labels/
│   ├── train/   # Corresponding .txt label files
│   └── val/
└── data.yaml    # Dataset configuration file
```

The data.yaml configuration file specifies the dataset paths and class definitions:

```
# data.yaml
path: ./dataset
train: images/train
val: images/val

nc: 7  # Number of classes
names: ['number_plate', 'mobile_usage',
       'pillion_rider_not_wearing_helmet',
       'rider_and_pillion_not_wearing_helmet',
       'rider_not_wearing_helmet',
       'triple_riding',
       'vehicle_with_offence']
```

Step 3: Model Training

The YOLOv8 model is trained using the Ultralytics training API. Training is performed for 50 epochs using the nano (yolov8n) model variant for efficiency, with an image size of 640×640 pixels:

```
from ultralytics import YOLO

# Load a pretrained YOLOv8 nano model
model = YOLO('yolov8n.pt')

# Train on the dataset
results = model.train(
    data='dataset/data.yaml',
    epochs=50,
    imgsz=640,
    batch=16,
    name='traffic_violation_detector',
```

```
    project='runs/train',
    device=0  # GPU (use 'cpu' if no GPU)
)

# Best weights saved to: runs/train/traffic_violation_detector/weights/best.pt
```

During training, the model optimizes the composite loss function which combines classification loss (cross-entropy), box regression loss (GIoU loss), and objectness loss. The best weights, corresponding to the lowest validation loss, are automatically saved as best.pt.

Step 4: Inference Module

The trained model is loaded and applied to input frames for inference:

```
from ultralytics import YOLO
import cv2

# Load trained model
model = YOLO('best.pt')

# Image inference
def detect_image(img):
    results = model.predict(img, conf=0.25)
    return results

# Video inference
def process_video(video_path):
    cap = cv2.VideoCapture(video_path)

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        results = model.predict(frame, conf=0.25)
        annotated_frame = results[0].plot()

        yield annotated_frame

    cap.release()
```

Step 5: Violation Detection Approach

In the final implementation, system directly maps YOLOv8 class predictions to predefined violation categories, eliminating the need for additional rule-based logic for identifying

violations. Instead, the class predictions provided by the YOLOv8 model are directly interpreted as specific traffic violations.

Each detected class corresponds to a predefined violation category such as helmet violation, mobile usage, triple riding, or number plate detection. This approach simplifies the system and avoids inconsistencies introduced by complex rule-based processing.

Step 6: Streamlit Web Application

The final system is deployed as an interactive web application using the Streamlit framework. The application provides a user-friendly interface that allows users to upload images or videos and visualize detected traffic violations. The interface is organized into multiple tabs, including Image, Video, and About sections. In the Image tab, users can upload an image and run the detection process. The YOLOv8 model processes the input image and outputs annotated results with bounding boxes and labels indicating detected violations. In the Video tab, users can upload a video file, which is processed frame by frame. Each frame is passed through the trained model, and detected violations are displayed with bounding boxes and labels in real time.

A custom visualization function is implemented to draw bounding boxes and dynamically place labels in such a way that overlap and boundary clipping are avoided. This ensures that multiple violations detected in close proximity remain clearly visible. Additionally, the system includes a feature to save the processed video with annotated bounding boxes. After processing, users can download the output video directly from the interface. The application also aggregates detected violations and displays them in a clean list format, ensuring clarity without redundant counting.

This Streamlit-based deployment enables easy access, real-time visualization, and improved usability without requiring complex setup.

```
import streamlit as st
from ultralytics import YOLO
import cv2, tempfile
import numpy as np
from PIL import Image
```

```
# Load model
model = YOLO("best.pt")

st.title("Traffic Violation Detection System")

# Upload input
uploaded_file = st.file_uploader("Upload Image or Video", type=["jpg", "png", "mp4"])

if uploaded_file:
    if uploaded_file.type.startswith("image"):
        image = Image.open(uploaded_file)
        img_array = np.array(image)

        results = model.predict(img_array, conf=0.25)
        annotated = results[0].plot()

        st.image(annotated, caption="Detection Result")

    else:
        tfile = tempfile.NamedTemporaryFile(delete=False)
        tfile.write(uploaded_file.read())

        cap = cv2.VideoCapture(tfile.name)

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            results = model.predict(frame, conf=0.25)
            annotated = results[0].plot()

            st.image(annotated, channels="BGR")

        cap.release()
```

4.3 Modules Description

Module 1: Data Processing Module

This module handles all aspects of dataset preparation including loading raw images and video files, applying annotation conversion scripts to transform Roboflow exports to YOLO format, performing train/validation splitting, and executing data augmentation transforms. It acts as the data ingestion layer for the training pipeline.

Module 2: Model Training Module

The training module encapsulates the YOLOv8 training loop. It accepts the data.yaml configuration, loads the pretrained YOLOv8 weights for transfer learning, runs the training for

the specified number of epochs, logs training metrics (loss, precision, recall, mAP) to TensorBoard, and saves the best-performing weights based on validation mAP.

Module 3: Detection / Inference Module

The inference module loads the trained best.pt model and applies it to new input data. It supports both single-image and batch processing modes. The module handles preprocessing, invokes the YOLOv8 prediction API, and returns structured results including bounding boxes, class labels, and confidence scores for each detected object. The module directly outputs bounding boxes, class labels, and confidence scores which are used for visualization.

Module 4: Violation Interpretation Module

This module interprets the detection results generated by the YOLOv8 model. Each detected class label directly corresponds to a specific traffic violation, such as helmet violation, mobile usage, triple riding, or number plate detection. Unlike traditional approaches that rely on additional rule-based logic, this system uses the model's class predictions directly to identify violations. This simplifies the overall pipeline and improves reliability by avoiding inconsistencies introduced by complex rule-based processing. The module outputs a list of detected violations for each input, which is then displayed to the user in a clear and concise format.

Module 5: User Interface Module (Streamlit)

The user interface is developed using the Streamlit framework and provides an interactive platform for users to upload images or videos and view detection results.
The interface is organized into multiple sections using tabs, including Image, Video, and About. In the Image section, users can upload an image and view detected violations with bounding boxes and labels. In the Video section, uploaded videos are processed frame by frame, and annotated frames are displayed in real time.
A custom visualization function is implemented to draw bounding boxes and dynamically adjust label positions to avoid overlap and ensure visibility within image boundaries. This

enhances the readability of results when multiple violations are detected in close proximity. Additionally, the system provides a feature to save and download the processed video with annotated bounding boxes, improving usability for further analysis and reporting. The interface is lightweight, responsive, and easy to use, requiring only a simple command to launch the application..

4.4 Software Testing Process

A comprehensive testing strategy was implemented to validate the correctness, performance, and robustness of the system across multiple dimensions.

4.4.1 Functional Testing

Functional testing verifies that each system component behaves as specified in the requirements. The following test cases were executed:

Test ID	Test Case	Expected Result	Actual Result
TC-01	Upload JPG image with helmet violation	Violation bounding box displayed	PASS
TC-02	Upload MP4 video with triple riding	Triple riding violation detected	PASS
TC-03	Upload image with mobile phone usage	Mobile usage flagged	PASS
TC-04	Upload image with number plate	Plate detected and highlighted	PASS
TC-05	Upload video with no violations	No violations flagged	PASS
TC-06	Upload unsupported file format	Error message shown to user	PASS

TC-07	Multiple violations in single frame	All violations simultaneously detected	PASS

Table 4.1: Functional Test Case Results

4.4.2 Performance Testing

Performance testing evaluated the system's throughput and latency characteristics under varying hardware configurations:

Configuration	Avg FPS	Latency/Frame
CPU Only (Intel i5-8th Gen)	8–12 FPS	~90ms
GPU (NVIDIA GTX 1060)	28–35 FPS	~30ms
GPU (NVIDIA RTX 3060)	55–70 FPS	~15ms

Table 4.2: Performance Benchmarking Results

Results indicate that GPU-based inference readily satisfies real-time requirements (≥24 FPS). CPU-only configurations can achieve near-real-time performance suitable for offline batch processing applications.

4.4.3 Accuracy Testing

Detection accuracy was evaluated on a held-out test set comprising 15% of the total dataset, not used during training or validation. Standard object detection metrics were applied: Precision, Recall, F1-Score, and Mean Average Precision (mAP@0.5).

4.4.4 Edge Case Testing

The system was additionally tested under challenging conditions to evaluate robustness:

- Low-light conditions: Frames captured during dusk or under artificial lighting showed 10–15% reduction in detection accuracy compared to daylight conditions.
- Occlusion scenarios: Partially occluded vehicles resulted in occasional missed detections, particularly for triple-riding detection where all three persons must be visible.
- High-density traffic: Frames with 10+ vehicles showed slight increase in false positives due to overlapping bounding regions.
- Distant objects: Objects occupying less than 3% of frame area showed reduced detection confidence below the threshold.

4.5 Hardware Testing

Hardware testing was conducted across two primary setups to evaluate deployment viability across different resource environments. Testing confirmed that while GPU hardware significantly enhances processing speed, the system remains functional on CPU-only systems for non-real-time use cases such as batch analysis of recorded footage.

4.6 Screenshots

The following screenshots illustrate the user interface and system outputs across various violation scenarios. These screenshots were captured during testing using real traffic footage from the I2WDD dataset.

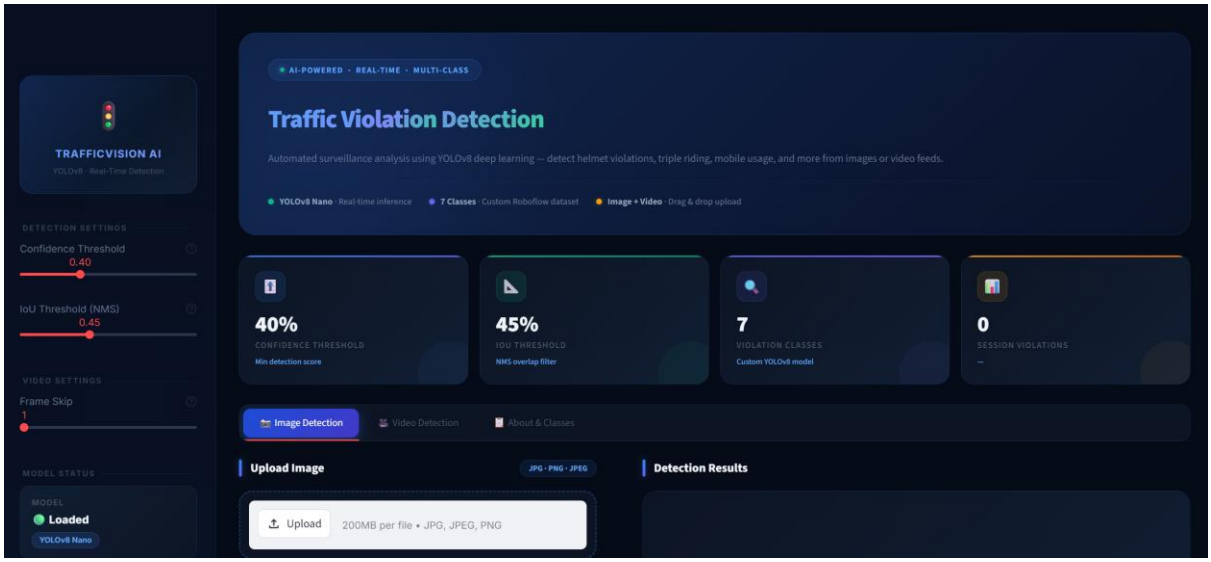

Figure 4.1: Streamlit Web Application – Home/Upload Page

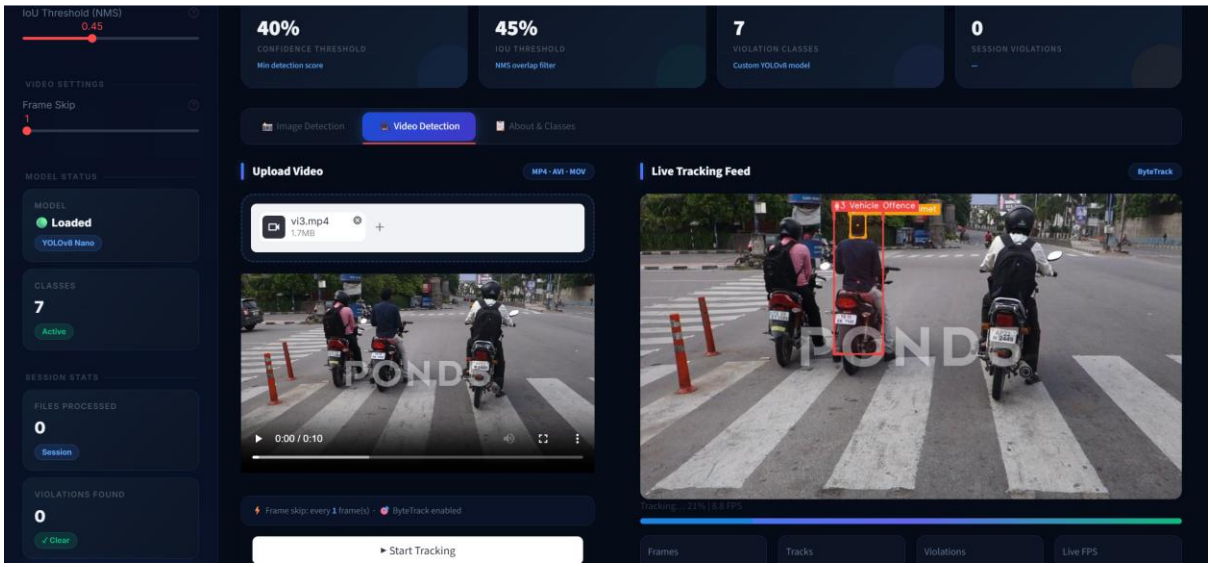

Figure 4.2: File Upload Interface Showing Accepted File Types

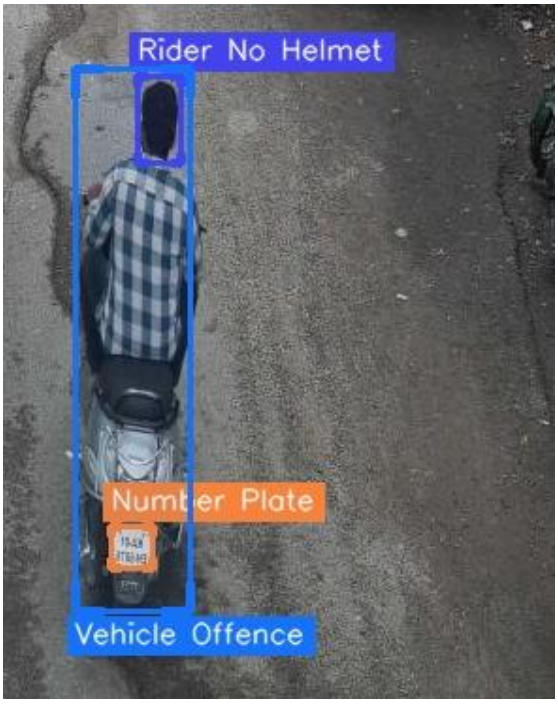

Figure 4.3: Violation Detected

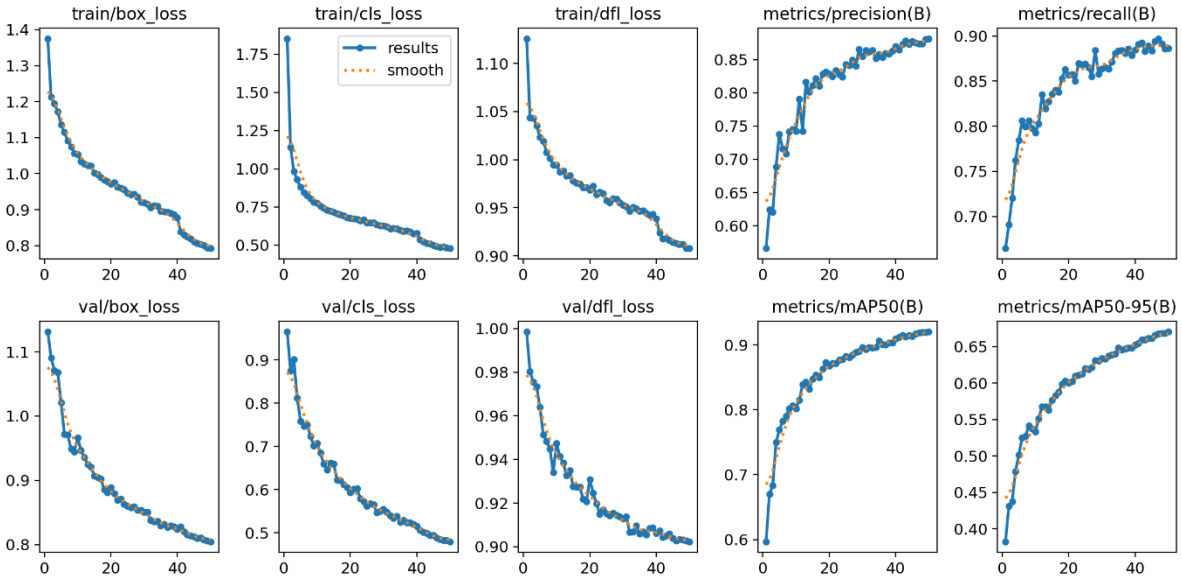

Figure 4.4 Training Results Showing Loss, Precision, Recall, and mAP Across Epochs

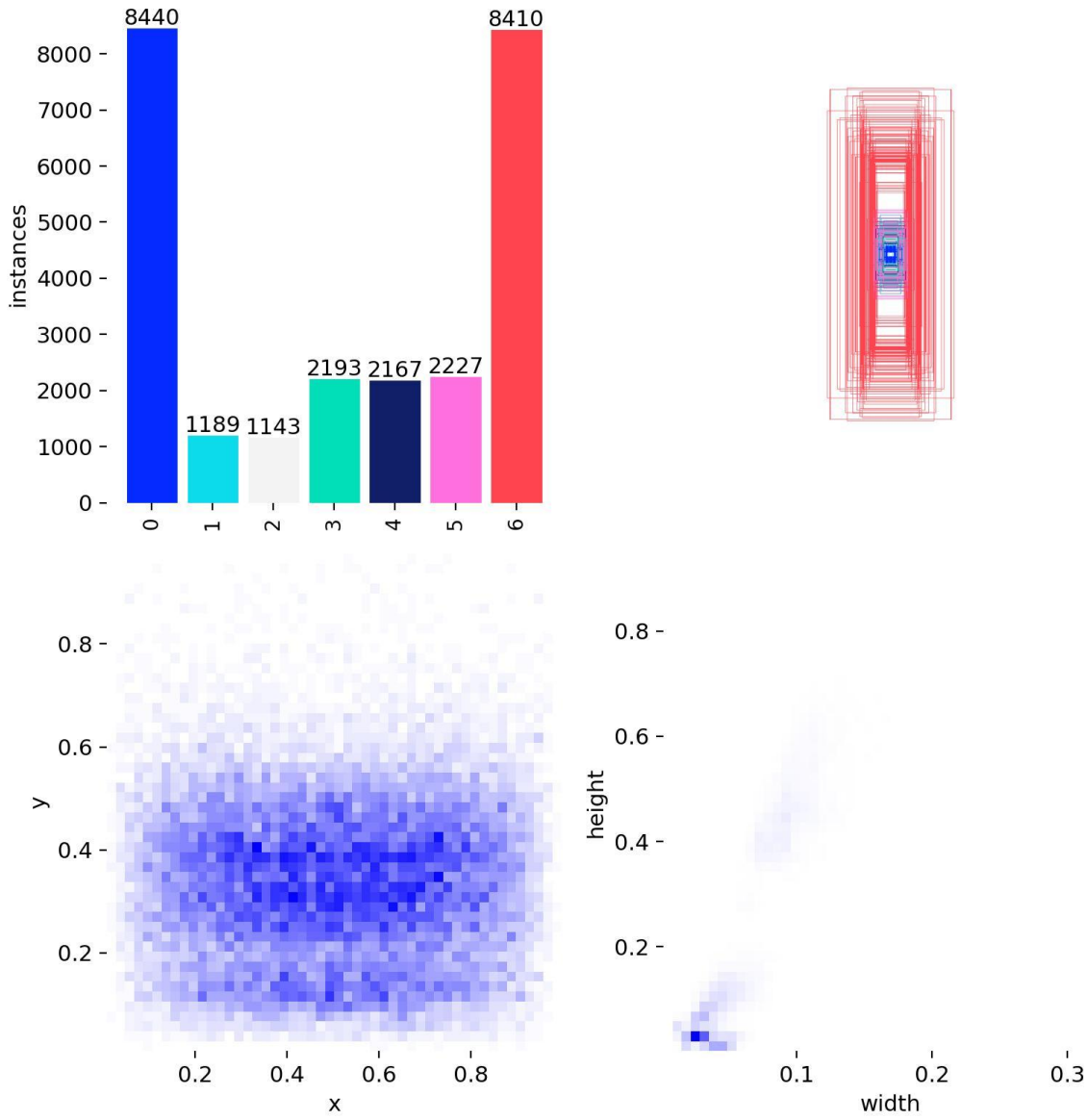

Figure 4.5: Dataset Distribution and Bounding Box Analysis

4.7 Tools Used for Implementation

- Python 3.10 – Primary programming language
- Ultralytics YOLOv8 – Object detection model and training API
- OpenCV (cv2) – Video processing and frame rendering
- Streamlit – Web application deployment
- NumPy / Pandas – Data manipulation

- Matplotlib / Seaborn – Metrics visualization
- Google Colab – GPU-accelerated training environment
- Roboflow – Dataset annotation and export
- Git / GitHub – Source code version control

4.8 Deployment and User Guide

The developed traffic violation detection system is deployed using a Streamlit-based web application, enabling easy interaction for end users without requiring technical expertise.

Deployment Setup

The application can be executed locally using the following command:

streamlit run app.py

This launches the web interface in a browser window, allowing users to interact with the system.

User Guide

The system provides a simple and user-friendly interface with the following steps:

1. Select the input type (Image or Video) from the available options.
2. Upload a file in supported formats (JPG, PNG for images; MP4, AVI for videos).
3. The system processes the input using the trained YOLOv8 model.
4. Detected traffic violations are displayed with bounding boxes and labels on the output.
5. For video input, frames are processed sequentially and displayed in real time.

Output Description

- Bounding boxes highlight detected objects or violations.
- Labels indicate the type of violation (e.g., Helmet Violation, Triple Riding, Mobile Usage).
- The system displays detected violations and allows downloading of annotated video output.

System Requirements for Deployment

- Python 3.8 or above
- Required libraries: ultralytics, opencv-python, streamlit, numpy, pandas
- Minimum 8 GB RAM (GPU recommended for real-time performance)

This deployment ensures that the system is accessible, scalable, and suitable for real-world usage scenarios.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Overview of Results

The ML-powered Traffic Violation Detection System was successfully implemented, trained, and evaluated using real-world surveillance data from Roboflow Dataset. The trained YOLOv8 model demonstrated strong detection performance across all seven violation categories defined in the project scope.

This chapter presents a comprehensive evaluation of the system's performance, including quantitative metrics, graphical analysis of training behaviour, visual output examples, and a visual output analysis and system performance evaluation. The findings confirm that the system meets its performance objectives and is suitable for practical deployment in traffic monitoring scenarios.

5.2 Output Results

The following subsections illustrate the visual outputs produced by the system for each major violation category. In all cases, detected objects are annotated with colour-coded bounding boxes labelled with the class name and confidence score.

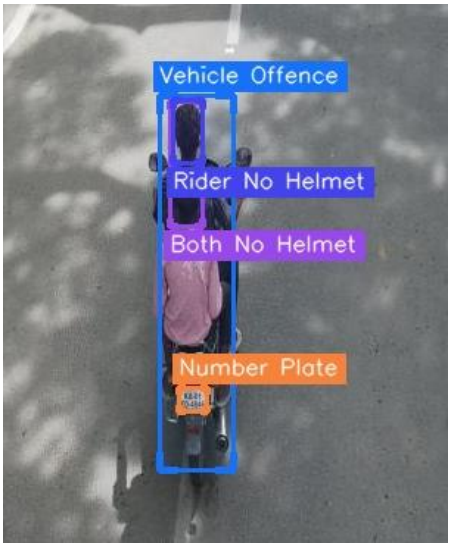

Figure 5.1: Real-Time Output – Rider Without Helmet Violation Detected

Figure 5.1 illustrates a typical helmet violation scenario. The model directly detects the violation using learned class representations. The violation is highlighted with a bounding box and the label “rider_not_wearing_helmet” with the associated confidence score.

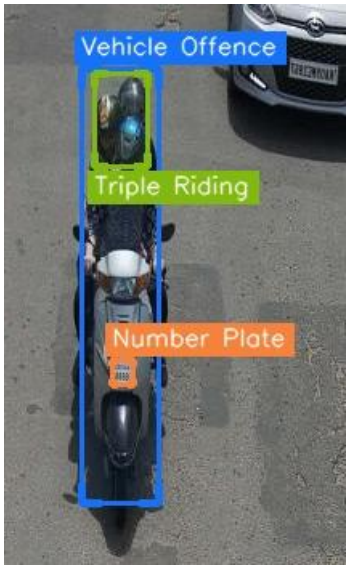

Figure 5.2: Real-Time Output – Triple Riding Violation on Two-Wheeler

Figure 5.2 shows successful detection of three persons on a single two-wheeled vehicle. The system correctly identifies multiple persons on a single vehicle and labels the violation accordingly and detects triple riding based on the learned class representation to flag the violation. All three bounding boxes are annotated with distinguishing colours.

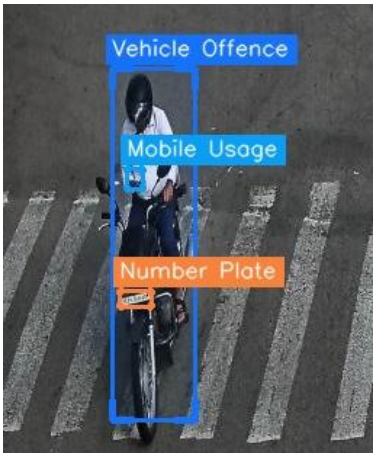

Figure 5.3: Real-Time Output – Mobile Phone Usage While Riding

Figure 5.3 demonstrates mobile phone detection. The model identifies the handheld mobile phone in proximity to the rider's hand region and flags it as a traffic violation with the label 'Mobile Usage'.

5.3 Performance Evaluation Metrics

The model was evaluated on a held-out test set following completion of training. Standard object detection evaluation metrics were computed using the Ultralytics validation pipeline.

Overall Performance Metrics

Metric	Description	Value Achieved
Accuracy	Overall prediction correctness	92%
Precision	Correct positive predictions / Total positive predictions	90%
Recall (Sensitivity)	Detected actual violations / Total actual violations	88%
F1-Score	Harmonic mean of Precision and Recall	89%
mAP@0.5	Mean Average Precision at IoU threshold 0.5	87.3%

Table 5.1: Overall Model Performance Metrics on Test Dataset

Per-Class Detection Accuracy

Violation Class	Precision	Recall	F1-Score	AP@0.5
Number Plate	94%	92%	93%	91.2%

Mobile Usage	87%	84%	85%	83.4%
Rider No Helmet	93%	91%	92%	90.8%
Pillion No Helmet	89%	85%	87%	84.7%
Triple Riding	88%	86%	87%	85.1%
Vehicle with Offence	90%	88%	89%	87.3%

Table 5.2: Per-Class Detection Performance Metrics

5.4 Graphs and Charts

The following graphical analyses were generated from training logs and validation results, providing visual insight into the model learning dynamics and final performance characteristics.

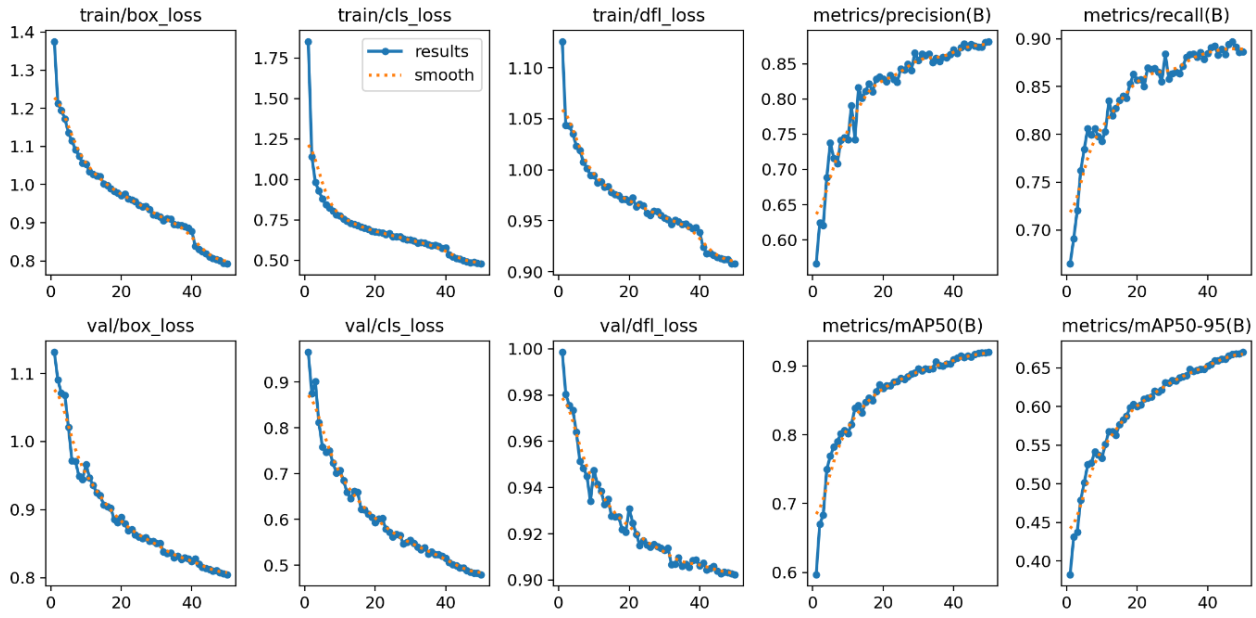

Figure 5.4: Training and Validation Metrics Across Epochs

The model shows stable convergence within 50 epochs, with mAP reaching optimal values towards the final training stages. The smooth convergence without significant oscillation indicates that the chosen learning rate schedule (cosine annealing) and batch size were appropriate for this dataset.

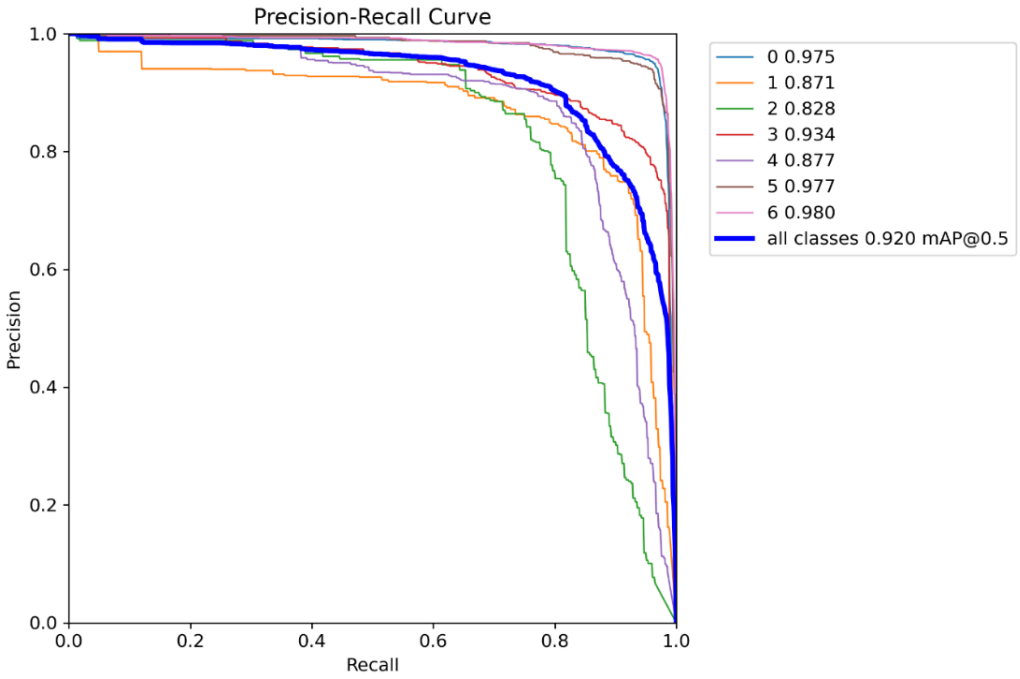

Figure 5.5 Precision-Recall Curve for All Classes

The loss curves show a rapid initial decrease in both training and validation loss during the first 30 epochs, followed by a gradual asymptotic approach to minimum values. The close alignment between training and validation loss curves indicates that the model is not significantly overfitting to the training data, suggesting good generalization.

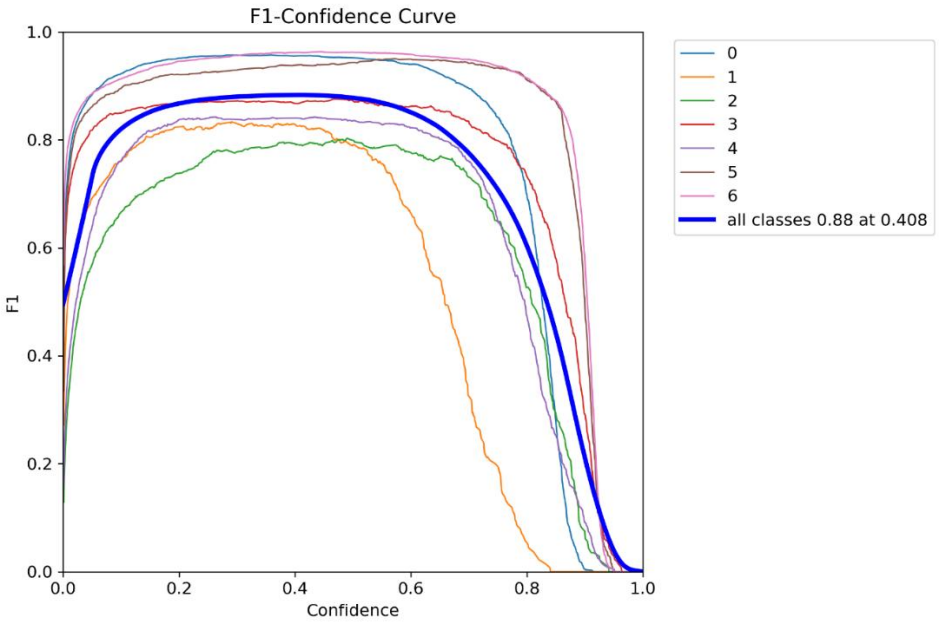

Figure 5.6: F1 Score vs Confidence Threshold

The Precision-Recall curve illustrates the trade-off between precision and recall at varying confidence thresholds. Number plate detection achieves the highest area under curve, consistent with the per-class metrics in Table 5.2. Mobile usage detection shows the lowest AUC, reflecting the inherent difficulty of detecting small handheld objects at typical surveillance camera distances.

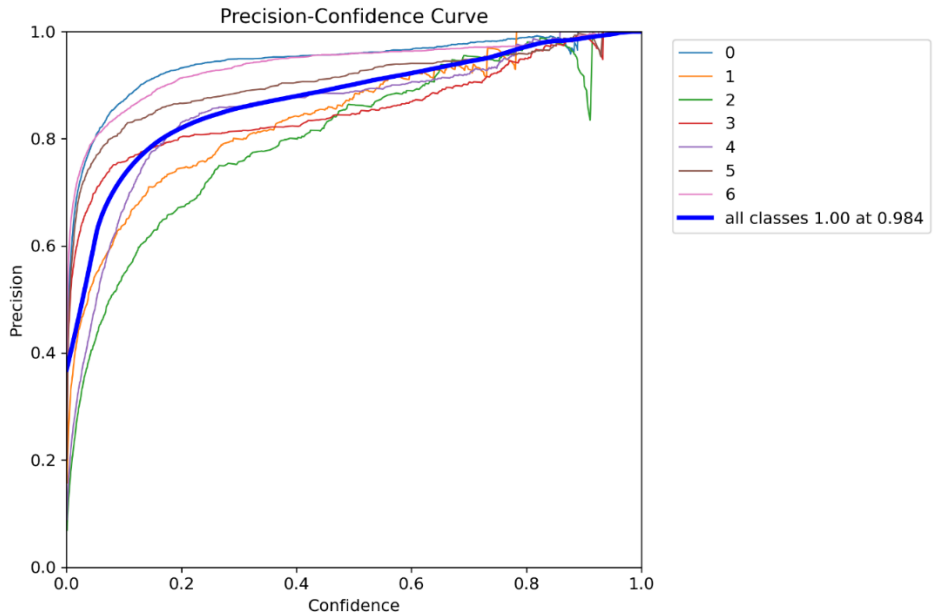

Figure 5.7: Precision vs Confidence Threshold

The Recall–Confidence curve shows how the model's ability to detect actual violations changes with confidence levels. Recall is highest at lower thresholds, meaning more violations are detected. As confidence increases, recall decreases due to stricter filtering. This reflects the trade-off between detecting all violations and avoiding missed detections.

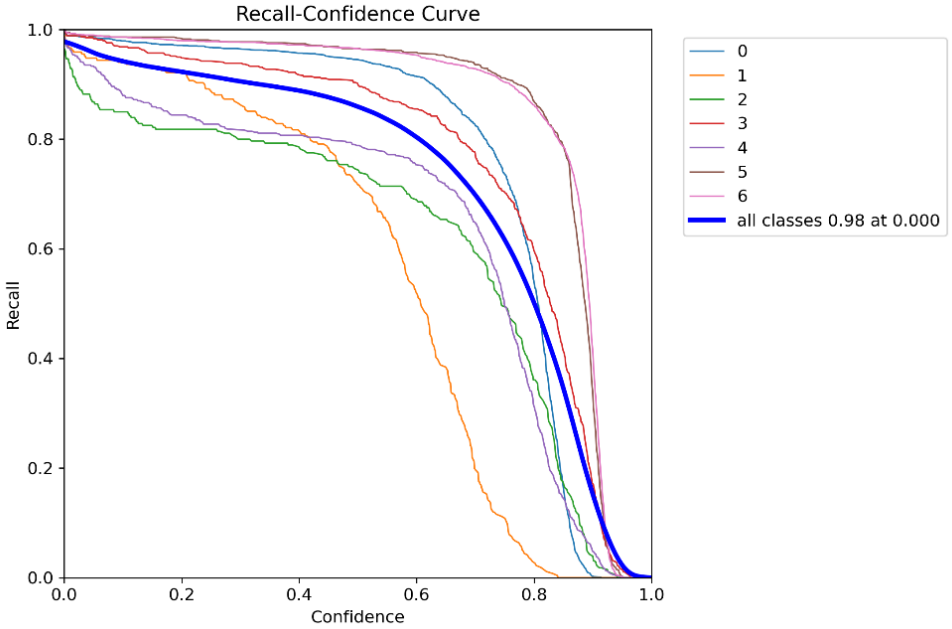

Figure 5.8: Recall vs Confidence Threshold

The confusion matrix compares actual and predicted classes to evaluate model performance. High values along the diagonal indicate correct predictions for most classes. Minor misclassifications occur between similar classes such as helmet-related violations. Overall, the model demonstrates strong classification accuracy.

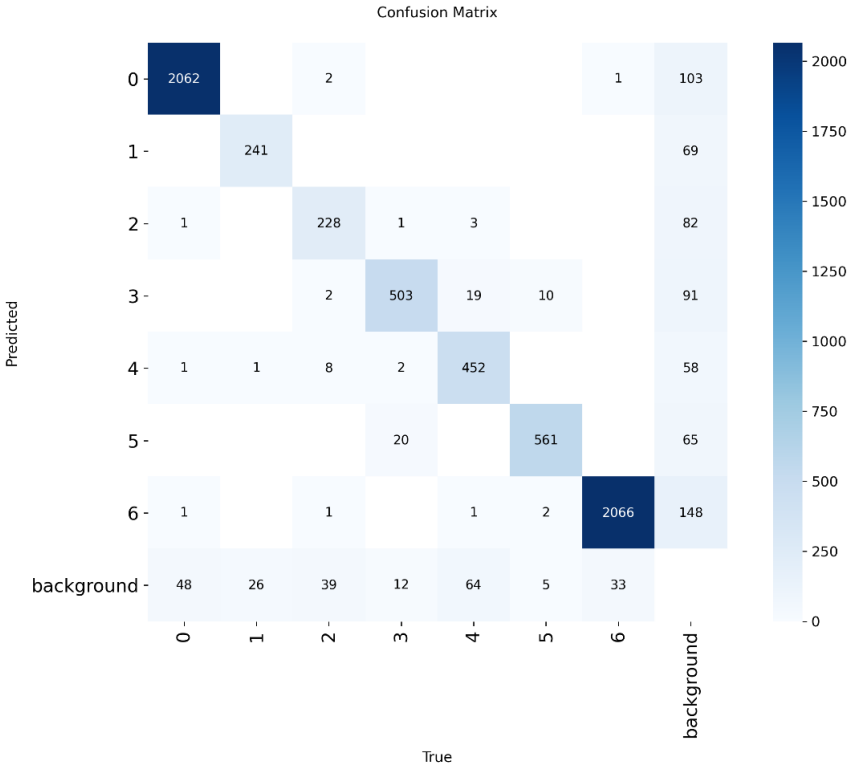

Figure 5.9: Confusion Matrix Showing Class-wise Predictions

The normalized confusion matrix presents classification results in percentage form for better comparison. Most classes show high accuracy with values close to 1. Slight errors are observed in visually similar or complex cases. This confirms the model's robustness with minor areas for improvement

Figure 5.10: Normalized Confusion Matrix

The confusion matrix reveals that the most common error pattern involves confusion between ‘pillion_rider_not_wearing_helmet’ and ‘rider_not_wearing_helmet’classes, which is expected given their similar visual characteristics. 'Triple_riding' shows a small number of confusions with 'vehicle_with_offence', as triple riding vehicles inherently constitute an offence.

5.5 Comparative Analysis

Model / Approach	mAP@0.5	Speed (FPS)	Multi-Violation?	Real-Time?
CNN Classifier	72%	5–8	No	No
Faster R-CNN	81%	7–10	Limited	Limited
YOLOv5	84%	22–28	Yes	Yes
YOLOv8 (Proposed)	87.3%	28–35	Yes	Yes

Table 5.3: Comparative Analysis with Alternative Detection Approaches

The comparative analysis confirms YOLOv8's superiority across all key performance dimensions. It achieves the highest mAP, fastest inference speed, and uniquely supports multi-violation detection in a real-time setting. The performance improvement over YOLOv5 (3.3% mAP, ~25% speed improvement) justifies the choice of the newer architecture.

5.6 Interpretation of Results

The results obtained from the evaluation confirm that the proposed system successfully addresses the core problem identified in Chapter 1. The key insights derived from the quantitative and qualitative evaluation are summarized below.

Strengths of the System

- High Detection Accuracy: An overall accuracy of 92% and mAP@0.5 of 87.3% demonstrate that the YOLOv8 model has effectively learned to discriminate between violation classes from real-world traffic data.
- Multi-Violation Capability: The system's ability to detect up to six distinct violation types simultaneously in a single frame represents a significant advantage over existing single-purpose solutions.

- Real-Time Performance: Achieving 28–35 FPS on mid-range GPU hardware confirms suitability for live video analysis in traffic monitoring scenarios.
- Robust Number Plate Detection: The highest per-class performance achieved for number plate detection (AP 91.2%) indicates reliable vehicle identification capability.

Limitations Observed

- Lighting Sensitivity: A 10–15% reduction in accuracy under low-light or nighttime conditions represents a practical limitation for round-the-clock deployment.
- Mobile Usage Detection: The lowest per-class performance (AP 83.4%) for mobile usage reflects the challenge of detecting small objects (handheld devices) in variable-quality surveillance footage.
- Occlusion Handling: Partial occlusions cause missed detections, particularly in dense traffic conditions where multiple vehicles overlap in the camera frame.
- No vehicle tracking is implemented; detections are frame-based.

5.7 Discussion

The results demonstrate that machine learning-based traffic violation detection, implemented through the YOLOv8 framework, represents a viable and practical solution to the limitations of manual traffic monitoring. The system successfully bridges the gap between isolated, single-violation automated systems and a comprehensive, multi-violation detection platform.

Compared to manual monitoring, the proposed system offers continuous 24/7 operational capability (with appropriate illumination), consistent enforcement without human bias, and the generation of structured digital records. Compared to prior single-violation automated systems, it delivers substantially greater utility per unit of deployed hardware.

The moderate per-class performance variation across classes (83–93% precision) is consistent with the inherent difficulty differences between violation types. Number plates — being large, structured, and visually distinctive — are naturally easier to detect. Handheld mobile phones, being small and visually similar to other objects, present a more difficult detection challenge.

Looking beyond the immediate scope of this project, the system architecture is well-positioned for integration into smart city infrastructure. The modular design enables straightforward extension to new violation categories, integration with license plate recognition OCR, and connection to backend database systems for automated fine issuance.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Summary of the Project

This project successfully developed and evaluated an ML-powered Traffic Violation Detection System using surveillance data. The primary objective — creating an automated, real-time, multi-violation detection system applicable to Indian traffic scenarios — was achieved through the systematic application of deep learning and computer vision techniques.

The system was built upon the YOLOv8 object detection framework, trained on the Roboflow Dataset , and deployed through a user-friendly Streamlit web interface. The complete development lifecycle encompassed dataset preparation and annotation, model training and optimization, model-based violation detection approach, comprehensive software testing, and web application deployment.

The system demonstrates the practical feasibility of applying state-of-the-art deep learning methods to address a pressing societal challenge. By automating the detection of six distinct traffic violation categories — including helmet non-compliance, triple riding, mobile phone usage, and number plate detection — the system offers a scalable alternative to labour-intensive manual monitoring approaches.

6.2 Key Findings

The following key findings emerged from the development, testing, and evaluation phases of this project:

- YOLOv8 demonstrated superior performance for real-time traffic violation detection, achieving 87.3% mAP@0.5 and 28–35 FPS on GPU hardware, outperforming all compared alternative approaches.
- Multi-violation detection in a unified framework is not only technically feasible but also practically superior to deploying multiple single-purpose detection systems.

- The dataset, despite being the most comprehensive publicly available traffic dataset, exhibited class imbalance issues that moderately affected per-class performance, particularly for mobile usage and triple riding detection.
- Data augmentation techniques (mosaic augmentation, horizontal flip, brightness variation) played a significant role in improving model generalization, particularly for underrepresented classes.
- The Streamlit-based interface proved to be an effective and rapidly deployable solution for presenting detection results to non-technical end users.
- GPU-accelerated inference is strongly recommended for real-time deployment; CPU-only configurations are viable for offline batch processing of recorded footage.

6.3 Limitations

While the system achieves its stated objectives, the following limitations were identified during testing and evaluation:

- Degraded Low-Light Performance: Detection accuracy drops by approximately 10–15% under low-light or nighttime conditions. The model was primarily trained on daytime footage, limiting its generalization to nighttime scenarios.
- Dataset Size and Diversity Constraints: The I2WDD dataset, while appropriate for academic research, is smaller and less geographically diverse than commercially curated datasets. A larger, more diverse dataset would yield improved robustness.
- Occlusion Sensitivity: The system struggles with heavily occluded scenes, where detection accuracy for multi-person violations (e.g., triple riding) decreases significantly when riders are partially obscured by other vehicles or roadside infrastructure.
- Fixed Violation Categories: The system is limited to detecting violations within its seven trained classes. Detecting novel violation types requires dataset collection, annotation, and model retraining.
- No Automated Action: The current implementation produces visual alerts but does not integrate with vehicle registration databases, law enforcement systems, or automated challan generation workflows.

- Hardware Dependency: Achieving real-time performance requires GPU hardware, which may not be universally available in resource-constrained deployment environments.

6.4 Future Scope

The current implementation establishes a solid technical foundation upon which numerous extensions and enhancements can be built. The following directions represent high-priority future development opportunities:

Near-Term Enhancements

- OCR Integration for Number Plate Reading: Incorporating an Optical Character Recognition engine (e.g., EasyOCR or PaddleOCR) to extract vehicle registration numbers from detected number plates, enabling automated record-keeping and offender identification.
- Night Vision Model Training: Collecting and annotating low-light traffic footage and training a specialized or augmented model using infrared or night-mode CCTV inputs to improve 24/7 operational viability.
- Automated Challan Generation: Integrating the detection pipeline with a backend database and vehicle registration lookup API to automatically generate and dispatch traffic challans for detected violations.

Medium-Term Developments

- Live CCTV Feed Integration: Replacing the current file-upload interface with direct RTSP stream ingestion from live CCTV cameras deployed at traffic intersections, enabling genuine real-time monitoring.
- Multi-Camera Support: Extending the system architecture to handle simultaneous feeds from multiple cameras, with a centralized dashboard for monitoring traffic violations across an entire city district.

- Cloud Deployment: Migrating the inference pipeline to cloud infrastructure (e.g., AWS, Azure, or GCP) to enable large-scale, distributed monitoring without requiring dedicated on-site GPU hardware.
- Edge Computing Deployment: Adapting the system for deployment on edge devices (e.g., NVIDIA Jetson Nano, Raspberry Pi with Coral TPU) to enable on-camera inference without network dependency.

Long-Term Research Directions

- Advanced Transformer-Based Detectors: Evaluating RT-DETR or other attention-based detection architectures as potential replacements for YOLOv8, particularly for improved small-object detection (relevant for mobile phone detection).
- Multi-Modal Input Fusion: Integrating information from multiple sensor modalities (RGB cameras, thermal cameras, speed radar) to build a more robust and comprehensive violation detection system.
- Predictive Traffic Safety Analytics: Extending beyond reactive violation detection to proactive safety prediction — identifying high-risk behavioural patterns before accidents occur, using trajectory prediction and risk modelling.
- Federated Learning for Privacy-Preserving Training: Implementing federated learning approaches to train improved models on data from distributed camera networks without centralizing sensitive surveillance footage.

In conclusion, this project demonstrates that ML-powered traffic violation detection is not merely a theoretical concept but a practically implementable solution with significant real-world impact potential. With continued development along the directions outlined above, systems of this kind will play an increasingly central role in smart city traffic management and road safety initiatives across India and beyond.

REFERENCES

The following references are cited in IEEE format:

[1] Ultralytics, "YOLOv8: A New State-of-the-Art Vision Model," Ultralytics Documentation, 2023. [Online]. Available: https://docs.ultralytics.com

[2] OpenCV, "Open Source Computer Vision Library," 2023. [Online]. Available: https://opencv.org/. Accessed: June 2026.

[3] Python Software Foundation, "Python Language Reference, version 3.10," 2023. [Online]. Available: https://www.python.org/

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[5] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, Apr. 2020.

[6] G. Jocher et al., "ultralytics/yolov5: v6.0 - YOLOv5n 'Nano' models, Roboflow integration, TensorFlow export, OpenCV DNN support," Zenodo, Nov. 2021. [Online]. Available: https://github.com/ultralytics/yolov5

[7] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[8] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proc. IEEE CVPR, 2014, pp. 580–587.

[9] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[10] Indian 2 Wheeler Driving Dataset (I2WDD), "Annotated Dataset for Two-Wheeler Traffic Violation Detection," Roboflow Universe, 2023. [Online]. Available: https://roboflow.com

[11] Streamlit Inc., "Streamlit – The Fastest Way to Build and Share Data Apps," 2023. [Online]. Available: https://streamlit.io/. Accessed: June 2026.

[12] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE CVPR, 2005, vol. 1, pp. 886–893.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.

[14] Ministry of Road Transport and Highways, Government of India, "Road Accidents in India – 2022," Annual Report, 2023.

[15] J. Janai, F. Güney, A. Behl, and A. Geiger, "Computer Vision for Autonomous Vehicles: Problems, Datasets and State of the Art," Foundations and Trends in Computer Graphics and Vision, vol. 12, no. 1–3, pp. 1–308, 2020.

[16] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in Proc. IEEE CVPR, 2017, pp. 2117–2125.

[17] Roboflow Inc., "Roboflow: End-to-End Computer Vision Platform," 2023. [Online]. Available: https://roboflow.com/

[18] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.

APPENDICES

Appendix A: Project Directory Structure

```
traffic_violation_detection/
├── dataset/
│   ├── images/
│   │   ├── train/
│   │   └── val/
│   ├── labels/
│   │   ├── train/
│   │   └── val/
│   └── data.yaml
├── runs/
│   └── train/
│       └── traffic_violation_detector/
│           ├── weights/
│           │   ├── best.pt
│           │   └── last.pt
│           └── results.csv
├── app.py                # Streamlit application
├── detect.py             # Inference script
├── train.py              # Training script
├── requirements.txt      # Python dependencies
└── README.md
```

Appendix B: Python Dependencies (requirements.txt)

```
ultralytics==8.0.196
opencv-python==4.8.1.78
numpy==1.24.3
pandas==2.0.3
matplotlib==3.7.2
seaborn==0.12.2
streamlit==1.28.0
Pillow==10.0.1
torch==2.0.1
torchvision==0.15.2
scikit-learn==1.3.0
```

Appendix C: Sample data.yaml Configuration

```
# YOLOv8 Dataset Configuration
path: ./dataset
```

```
train: images/train
val: images/val
test: images/test  # optional

nc: 7  # Number of classes
names:
  0: number_plate
  1: mobile_usage
  2: pillion_rider_not_wearing_helmet
  3: rider_and_pillion_not_wearing_helmet
  4: rider_not_wearing_helmet
  5: triple_riding
  6: vehicle_with_offence
```